हेलो! आपका स्वागत है **Unit 7: Strings** की complete study class में.

मैंने आपके workspace में, unit 7 strings folder के अंदर ये files create कर दी हैं:

- Study Guide File: chapter7_strings.md
- Code Examples File: strings_examples.js
- Practice Exercises File: strings_practice.js

चलिए अब JavaScript Strings को एकदम clear and simple Hinglish explanation के साथ समझना शुरू करते हैं.

PART 1: STRING BASICS

JavaScript में character streams को store करने के लिए strings का use किया जाता है.

1. String Kya Hai?

What is it? String text characters (letters, numbers, spaces, symbols) का collection block होता है.

How strings are created? JavaScript strings create karne ke 3 models follow karta hai:

- Single quotes: 'javascript'
- Double quotes: "javascript"
- Template literals: `javascript` (backticks keys)

Template literals basic Backticks use karne par hum variable injection expressions ${variable} utilize kar sakte hain. Is content insertion property ko String Interpolation bolte hain.

How JavaScript handles strings JavaScript strings internally immutable attributes properties behave karte hain. Iska matlab hai ki string contents modification direct perform nahi ho sakta. Jo bhi operations triggers apply karenge, woh ek naye location address output string create karenge.

---

2. String Indexing & Length

What is it?

- **Indexing:** String ka har single elements character index arrays coordinate position handle check (starts at index 0).
- **Length:** String total size variables check length parameters indicators.

Example Indexing dry run: String content: "CODE"

Character	C	O	D	E
Index position	0	1	2	3

Example Code

javascript

let target = "CODE";

console.log(target[0]); // 'C'

console.log(target.length); // 4

Important point Last character index calculation parameters check hamesha length - 1 coordinates.

Common mistakes Bracket indexing elements check modification code apply errors:

javascript

// Galti:

let val = "cool";

val[0] = "W"; // immutable variables output modification ignored

console.log(val); // cool

PART 2: STRING METHODS

1. charAt()

What is it? Diye gaye index value criteria matches parameters single characters values fetches.

Syntax string.charAt(index)

Simple example

javascript

let text = "React";

console.log(text.charAt(1));

Output e

Difference: charAt() vs normal indexing

- Normal indexing (text[100]) search parameters limits cross values returns undefined.
- charAt(100) target validation matches failure return safely empty string "".

---

2. includes()

What is it? Check rules search query substring inputs present true/false indicators checks.

Syntax string.includes(searchValue)

Simple example

javascript

let line = "Learn Javascript easily";

console.log(line.includes("Javascript"));

Output true

---

3. indexOf() vs lastIndexOf()

What is it?

- **indexOf():** Returns character query occurrence parameter first index matching position search start.
- **lastIndexOf():** Returns character matching index position from last occurrence search start.

Simple example

javascript

let target = "test code test";

console.log(target.indexOf("test"));

console.log(target.lastIndexOf("test"));

Output 0 10

Important point Searching matches parameter failure evaluates return criteria defaults to **-1**.

---

4. slice() vs substring()

What is it? Extracts character substrings elements within range parameters.

Difference details:

- **slice(start, end):** Negative index parameter numbers count handles from string end boundaries.
- **substring(start, end):** Negative argument conversions defaults directly to 0.

Example

javascript

```
let text = "Developer";

console.log(text.slice(-5)); // returns last 5 letters

console.log(text.substring(-3, 3)); // negative acts as 0, returns "Dev"
```

Output loper Dev

---

5. toUpperCase() & toLowerCase()

What is it? String case converters templates uppercase / lowercase operations.

Example

javascript

```
let value = "Code";

console.log(value.toUpperCase());

console.log(value.toLowerCase());
```

Output CODE code

---

6. trim()

What is it? Clean space coordinates start and end filters blocks space lines.

Important point Only clears outer edge white space variables list, middle string spaces coordinates matches ignores.

Example

javascript

```
let phrase = "  my app  ";

console.log(phrase.trim());
```

Output my app

---

7. replace() vs replaceAll()

What is it?

- **replace():** Updates only the first matching occurrence.
- **replaceAll():** Replaces all matching occurrences in the string.

Example

javascript

let testStr = "cat meets cat";

console.log(testStr.replace("cat", "dog"));

console.log(testStr.replaceAll("cat", "dog"));

Output dog meets cat dog meets dog

---

8. split() vs join()

What is it?

- **split():** Separator matching base properties check strings converted to Array elements.
- **join():** Merge array lists variables back to single string.

Example

javascript

let words = "one,two,three";

let splitArr = words.split(",");

console.log(splitArr);

let joinStr = splitArr.join(" ");

console.log(joinStr);

Output ["one", "two", "three"] one two three

PART 3: PRACTICAL STRING PROBLEMS

Reverse a String

javascript

```
function reverseStr(str) {
   return str.split("").reverse().join("");
}
console.log(reverseStr("hello")); // olleh
```

Check Palindrome

```
javascript
function isPalindrome(str) {
   let clean = str.toLowerCase().replace(/[^a-z0-9]/g, "");
   return clean === clean.split("").reverse().join("");
}
console.log(isPalindrome("racecar")); // true
```

Count Vowels

```
javascript
function getVowelCount(str) {
   let list = "aeiouAEIOU";
   let total = 0;
   for (let char of str) {
      if (list.includes(char)) total++;
   }
   return total;
}
console.log(getVowelCount("javascript")); // 3
```

Swap Case

```
javascript
function swapLetters(str) {
   let output = "";
   for (let char of str) {
      if (char === char.toUpperCase()) {
```

```
            output += char.toLowerCase();
        } else {
            output += char.toUpperCase();
        }
    }
    return output;
}
console.log(swapLetters("Code")); // cODE
```

🧠 QUICK REVISION SUMMARY

- **Immutability:** Original strings cannot be changed; methods return new strings.
- **slice vs substring:** slice handles negative values (counts from end); substring turns negative to 0.
- **replace vs replaceAll:** replace updates the first occurrence; replaceAll updates all of them.
- **charAt vs indexing:** charAt returns "" for out-of-range indices; indexing ([]) returns undefined.
- **split and join:** split cuts string to array; join binds array to string.

📝 IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **String Immutability:** The programming design trait ensuring a string's character values cannot be altered post-instantiation.
2. **String Interpolation:** Evaluating expressions within backtick strings using ${expression} wrappers.
3. **Array Tokenization:** Splitting a string into array substrings using matching delimiter sequences.
4. **Out-of-Bounds Evaluation:** Reading character values outside string boundaries, yielding undefined via brackets or "" via charAt().
5. **String Sanitization:** Using helper methods like trim() to filter and clean unwanted edge characters.

💬 **IMPORTANT INTERVIEW QUESTIONS & ANSWERS**

Q1. What is string immutability?

Ans: JavaScript strings cannot be changed after creation. String methods do not modify the original string but return a new one.

Q2. Difference between slice() and substring()?

Ans: slice() accepts negative index arguments to count from the end of the string. substring() treats negative indexes as 0.

Q3. What is the output of "apple"[10] and "apple".charAt(10)?

Ans: "apple"[10] yields undefined. "apple".charAt(10) returns an empty string "".

Q4. Predict output:

javascript

let text = "hello";

text[0] = "H";

console.log(text);

Ans: "hello". Strings are immutable, so assignments to specific index positions are ignored.

Q5. What does indexOf() yield if substring is not found?

Ans: It returns -1.

Q6. Difference between replace() and replaceAll()?

Ans: replace() modifies only the first matching instance. replaceAll() updates all matching occurrences.

Q7. How do you convert array ["x", "y"] into string "xy"?

Ans: By using join("") with an empty string as separator.

Q8. What does trim() remove?

Ans: It removes whitespace characters (spaces, tabs, newlines) only from the beginning and end of the string.

Q9. Predict output:

javascript

let str = "coder";

console.log(str.slice(-2));

Ans: "er". It returns the last two characters of the string.

Q10. What is string interpolation?

Ans: Embedding variable expressions directly within backtick template literals using ${variableName} syntax.

Q11. Predict output:

javascript

console.log("hello".includes("H"));

Ans: false. The includes() check is case-sensitive.

Q12. Predict output:

javascript

let s = "banana";

console.log(s.replace("a", "o"));

Ans: "bonana". Only the first match is replaced.

Q13. Predict output:

javascript

let word = "abc";

console.log(word.split("").reverse().join(""));

Ans: "cba". It splits the string into characters, reverses the array, and joins them back.

Q14. Predict output:

javascript

console.log(typeof ("5" + 5));

Ans: "string". The number is coerced to a string and concatenated, resulting in "55".

Q15. Predict output:

javascript

console.log("10" - "2");

Ans: 8. The mathematical subtraction operation coerces both strings to numbers.

Q16. What is the difference between indexOf() and lastIndexOf()?

Ans: indexOf() searches from the beginning and returns the first match index. lastIndexOf() searches from the end and returns the last match index.

Q17. Predict output:

javascript

```
let x = "Code";

console.log(x.slice(0, 1));
```

Ans: "C". The end index is exclusive.

Q18. Predict output:

javascript

```
console.log(" ".length);
```

Ans: 1. Space characters are counted as valid elements.

Q19. Can you change string variable reference declared with let?

Ans: Yes. The variable reference can be reassigned, even though the string values themselves are immutable.

Q20. Predict output:

javascript

```
let a = "cat";

console.log(a.substring(1, 1));
```

Ans: "". If start and end indexes are equal, it returns an empty string.

💻 PRACTICE QUESTIONS

Question 1: Count Vowels inside string

- **Question:** Create a function to find vowel counts.
- **Solution:**

javascript

```
function findVowels(str) {
  let match = "aeiouAEIOU";
  let count = 0;
```

```
  for (let char of str) {
    if (match.includes(char)) count++;
  }
  return count;
}
console.log(findVowels("developer")); // 4
```

- **Explanation:** Loops through characters and increments counter if they exist in vowels list.

Question 2: Swap casing format

- **Question:** Shift lower to uppercase and uppercase to lowercase.
- **Solution:**

```
javascript
function casingShift(str) {
  let output = "";
  for (let char of str) {
    if (char === char.toUpperCase()) {
      output += char.toLowerCase();
    } else {
      output += char.toUpperCase();
    }
  }
  return output;
}
console.log(casingShift("ProJS")); // pROjs
```

- **Explanation:** Compares letter case representation and appends swapped case output.

Question 3: Remove string space gaps

- **Question:** Remove all whitespace characters.
- **Solution:**

javascript

```
function deleteGaps(str) {
    return str.split(" ").join("");
}
console.log(deleteGaps("x y z")); // xyz
```

- **Explanation:** Tokenizes characters by space delimiter, then joins array items. }